

Knowledge Graph Completion Analysis

Knowledge Graph Completion Analysis

Scope

The coursework brief requires a completion analysis that identifies gaps in the knowledge graph and proposes a retrieval-augmented strategy for filling them. This document evaluates the current repository snapshot rather than an idealized target system. The main finding is that the graph already answers all 20 stored competency questions, but it still contains important ontology-level and instance-level incompleteness that should be addressed before claiming full automation and full semantic coverage.

Summary Of Current State

- Final merged graph size: 17,539 triples
- Stored SPARQL competency questions answered: 20/20
- Canonical coursework namespace: `http://example.org/tfl#`
- Additional disconnected namespaces:
 - Linux file-based `file:///.../ontologies/pipeline_output/tfl#`
 - Windows file-based `file:///.../ontologies/pipeline_output/tfl#`
- Structured file-namespace subjects: 3,293
- Unstructured file-namespace subjects: 2,273

This means the repository contains a large amount of harvested information that is not currently contributing to the main coursework query namespace.

Incomplete Ontology Elements

1. Canonical namespace model

Evidence in repo: manual ontology uses `http://example.org/tfl#`, while pipeline outputs use file-based namespaces.

Why it is incomplete: the project currently behaves like three related KGs rather than one integrated KG.

Completion strategy: standardize every generator on a single base IRI and regenerate all outputs.

2. Transport mode abstraction

Evidence in repo: CQ3 is framed in terms of transport modes, but the current queryable graph mainly returns line or service names.

Why it is incomplete: the ontology lacks a consistent `TransportMode` layer for Tube, Bus, DLR, Elizabeth line, and Overground.

Completion strategy: introduce a `TransportMode` class and a property such as `hasAvailableMode`, then materialize mode instances.

3. Rule-based class materialization

Evidence in repo: `isNightTube true` exists for five lines, but only four are typed as `NightTubeLine`.

Why it is incomplete: boolean facts and class membership are not fully synchronized.

Completion strategy: add inference rules or a deterministic post-processing materialization step.

4. Temporal disruption model

Evidence in repo: only one engineering closure instance is represented, despite manifest support for status retrievals.

Why it is incomplete: the ontology does not yet capture repeated or refreshed closure snapshots as a general pattern.

Completion strategy: extend the disruption model with validity windows and snapshot provenance.

5. Provenance model across pipelines

Evidence in repo: unstructured triples carry `dcterms:source` and `prov:wasGeneratedBy`; structured and manual facts do not do so consistently.

Why it is incomplete: provenance is uneven, which weakens trust and traceability.

Completion strategy: apply the same provenance design to structured and manual enrichment layers.

Incomplete Instance Elements

1. Automated triples disconnected from the main namespace

Evidence in repo: 3,293 structured and 2,273 unstructured subjects live in file-based namespaces.

Why it matters: harvested data cannot be queried through the coursework namespace.

Completion strategy: rewrite generated IRIs to the canonical namespace and align duplicate labels or identifiers.

2. Night Tube classification inconsistency

Evidence in repo: five lines have `tfl:isNightTube true`, but only four are typed `tfl:NightTubeLine`.

Why it matters: semantic classification and query behavior can drift apart.

Completion strategy: add the missing class assertion for Central Line or derive it automatically.

3. Sparse terminal-station coverage

Evidence in repo: only one line currently has `tfl:hasTerminalStation` facts.

Why it matters: route-sequence data exists in the source manifest but is not fully materialized.

Completion strategy: generate termini from first and last stops of cached route sequences.

4. Missing raw text snapshots

Evidence in repo: `data/raw/...` is absent even though the unstructured pipeline expects it.

Why it matters: the textual extraction process cannot be replayed end-to-end from the committed repo alone.

Completion strategy: commit the snapshot text files or add a deterministic rebuild script with archived sources.

5. Local data-quality defects in manually seeded instances

Evidence in repo: examples include `BakerloLine`, `NorthGreenwickStation`, and `RoystonStation` representing other concepts or labels.

Why it matters: these defects reduce professional polish and make reconciliation harder.

Completion strategy: normalize instance identifiers and add validation tests for label and IRI consistency.

Retrieval-Augmented Completion Strategy

The most effective completion strategy for this project is a KG-centered RAG workflow with three retrieval layers:

1. KG retrieval: use SPARQL over the canonical graph to identify gaps, missing class assertions, sparse properties, and duplicate labels.
2. Source retrieval: use `docs/generated/tfl-sources.json`, `downloads/tfl_api_cache.json`, and the unstructured extraction cache to recover supporting evidence.
3. Completion generation: provide an LLM with the retrieved graph context plus source snippets and ask it to propose new triples, mappings, or alignments, which are then validated before insertion.

This is preferable to free-form prompting because the KG already holds strong domain anchors such as line labels, station labels, zone identifiers, operator names, and fare concepts. Retrieval constrains generation and lowers hallucination risk.

RAG Results On Three Worked Gaps

Case 1: Night Tube class completion

Retrieval from the current KG shows:

- `CentralLine tfl:isNightTube true`
- four explicit `NightTubeLine` instances
- CQ10 returning five Night Tube lines

This is a classic completion target for KG-backed generation. The retrieved graph context is enough to justify the candidate completion:

- `tfl:CentralLine rdf:type tfl:NightTubeLine`

Result: the KG can be made class-consistent with its own boolean service facts.

Case 2: Terminal-station enrichment

Retrieval from the source manifest shows 695 route-sequence fetches were planned and logged. Retrieval from the canonical graph shows only one line currently has explicit `hasTerminalStation` facts.

Candidate completion pattern:

- retrieve first and last stop from each cached route-sequence record
- map the stop to a canonical station IRI
- assert `tfl:hasTerminalStation`

Result: the same modeling pattern already used for the Victoria line can be extended across the wider network.

Case 3: Namespace alignment

Retrieval from the merged graph shows:

- 3,293 structured subjects in the Linux file-based namespace
- 2,273 unstructured subjects in the Windows file-based namespace
- only the manually curated `http://example.org/tfl#` namespace is used by the course-work SPARQL questions

Candidate completion pattern:

- retrieve entities with matching labels across namespaces
- align them to the canonical namespace by identifier or exact label match
- rewrite or map file-based predicates and classes into the core ontology namespace

Result: this single completion step would unlock a large volume of already harvested automated data for the main query layer.

Recommended Next Actions

1. Fix the namespace constant in both generation pipelines and rebuild all outputs.
2. Materialize inference-like facts such as Night Tube class membership deterministically after graph generation.
3. Use the logged route-sequence sources to generate terminal stations for all eligible lines.
4. Restore or commit the raw textual snapshots required by the unstructured pipeline.
5. Add validation tests for canonical IRIs, provenance coverage, and duplicate-label alignment.

Conclusion

The current graph is functionally useful but not complete in the sense intended by the course-work brief. Its biggest gap is not lack of harvested data, but lack of integration between the harvested data and the canonical ontology namespace. A KG-guided RAG workflow is well suited to this repository because the graph already contains enough structure to drive targeted completion, especially for namespace repair, missing classifications, sparse route facts, and provenance enrichment.